

Problem Statement

Task: signal $X \in \mathbb{R}^{C \times T} \rightarrow$ Action Prediction

Dataset

Physionet $\Rightarrow$ 109 subjects, 64 channels
 1 subject $\Rightarrow$ 14 runs $\Rightarrow$ same session (real/imagery)
 2 runs $\rightarrow$ eyes open / closed (baseline)
 4 runs $\rightarrow$ Task 1 (real $\Rightarrow$ left vs right fist)
 Task 2 (imagined $\Rightarrow$ left vs right fist)
 Task 3 (real $\Rightarrow$ both fists vs both feet)
 Task 4 (imagined $\Rightarrow$ both fists vs both feet)

$\times 2$ more times $\Rightarrow$ Total 14 runs.

80 total 6 real vs 6 imagery runs.

1 run = 2 min 58 $\Rightarrow$ 125s

each run $\Rightarrow$ 4s interval between actions

4s rest (T_0), 4s action (T_1/T_2)

Total 15 actions (7-8 T_1/T_2)

Sampling rate $\Rightarrow$ 160 Hz $\rightarrow$ 125s $\Rightarrow$ 20000 samples

$C \in \{1, 2, 3, \dots, 64\}$

$T \in \{1, 2, \dots, 20000\}$

BCIIV-2a: 9 subjects, 22 EEG + 3 EOG

1 subject: 2 sessions (T & E) $\Rightarrow$ cross session

Actions: Left hand, right hand, both feet, Tongue

each session: 288 trials $\Rightarrow$ 4x4 actions

each trial: 7-8s $\Rightarrow$ 2s cue, 4s action, rest

Sampling frequency = 250 Hz.

Total time / session = 288 x (7-8) $\approx$ 2000-2400s (T)

No. of samples: 250 x (2000-2400) = 500000 - 600000 samples

Dataset representation

$X_{i,a,e} \in \mathbb{R}^{C \times T_e}$; $i = 1, 2, 3, \dots, 109$ subjects, $a \in$ Action
 T_e = epoch time (4s in Physionet, 3s/4s after cue).

Preprocessing

RIEMANNIAN MANIFOLD.

Distance $\Leftrightarrow$ geodesic distance (4-5s)

$X_{i,a,e} \in \mathbb{R}^{C \times T_e}$; $C = \{1, 2, \dots, 64\}$, $T_e = \{1, 2, \dots, 7213\}$

$C_{i,a} = \frac{1}{C-1} X_{i,a,e} X_{i,a,e}^T \in \text{SPD}(C) \Rightarrow C \times C$ matrices

AIRM geodesic distance

$$d(C_1, C_2) = \|\log C C_1^{-1/2} C_2 C_1^{-1/2}\|_F$$

symmetric: $d(C_1, C_2) = d(C_2, C_1)$ — (i)

Mean (N_T : No of trials)

Riemannian Mean: $C_{R, \text{ref}, i} = \arg \min_{C} \sum_{e,a} d_R^2(C, C_{i,a,e})$
 of subject i

Euclidean Mean: $C_{E, \text{ref}, i} = \frac{1}{N_T} \sum_{e,a} C_{i,a,e}$ — (ii)

log-Euclidean Mean: $C_{LE, \text{ref}, i} = \frac{1}{N_T} \sum_{e,a} \log C_{i,a,e}$ — (iii)

Whitening

$C_{R, \text{ref}, i}$ is Riemannian mean of subject i .

Consider a matrix $C_{i,a,e}$ of subject i

$$C_{i,a,e}^{-1/2} C_{R, \text{ref}, i} C_{i,a,e}^{-1/2} \quad \text{--- (iv)}$$

$\Rightarrow$ We do this because, each subject i has $C_{R, \text{ref}, i}$ reference that is used to identify the subject. Therefore if C_{ref} of all subjects are I , then subjects are generalized & no cues for identifying subject

$$X'_{i,a,e} = C_{R, \text{ref}, i}^{-1/2} X_{i,a,e} \quad \text{--- (v)}$$

Here X' is normalized, $\sim x/\sigma$.

So $\mu/\sigma \Rightarrow$ deviation from I where all $x_i/\sigma \Rightarrow$ same μ . So this can be used specifically for classification.

RIEMANNIAN METHODS

Tangent Space Logistic Regression (CSP)

1. $X_{i,a,e}$, $a \in A$, $A \in \{0, 1, 2, 3\} \Rightarrow$ Physionet
 $e \in \{1, 2, \dots, N_T\}$

2. Compute Covariance $C_{i,a,e}$ of each $X_{i,a}$

$$C_{i,a,e} = \frac{1}{C-1} X_{i,a,e} X_{i,a,e}^T$$

3. Compute per subject C_{ref} .

Riemannian Mean: $C_{R, \text{ref}, i} = \arg \min_C \sum_e d_R^2(C, C_{i,e})$

Euclidean Mean: $C_{E, \text{ref}, i} = \frac{1}{N_T} \sum_e C_{i,e}$

Log-Euclidean Mean: $C_{LE, \text{ref}, i} = \frac{1}{N_T} \sum_e \log C_{i,e}$

4. Align the Covariances to the per subject ref such that C_{ref} of updated covariances per subject = I

We can use $C_{\text{ref}, i} = C_{R, \text{ref}, i}$, $C_{E, \text{ref}, i}$, $C_{LE, \text{ref}, i}$ depending on type of Alignment.

$$C'_{i,a,e} = C_{\text{ref}, i}^{-1/2} C_{i,a,e} C_{\text{ref}, i}^{-1/2}$$

5. Compute $C_{\text{ref}}^{\text{global}} = \arg \min_C \sum_{i,a,e} \|\log C^{-1/2} C'_{i,a,e} C^{-1/2}\|_F$

6. Project C' onto tangent space at $C_{g, \text{ref}}$

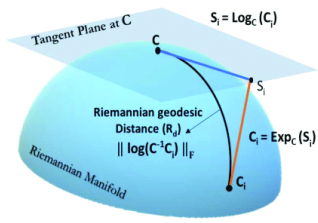
$$T_{i,a,e} = C_{g, \text{ref}}^{1/2} \log C C_{g, \text{ref}}^{-1/2} C'_{i,a,e} C_{g, \text{ref}}^{-1/2} C_{g, \text{ref}}^{1/2}$$

7. Perform Multinomial Logistic Regression.

8. $x_{i,a,e} = \text{vec}(T_{i,a,e})$

$$x_{(i-1) \times N + j} = \begin{cases} T_{ij} & i = j \\ \sqrt{2} T_{ij} & i \neq j \end{cases}$$

10. $z_k = w_k^T x_i + b_k$
 $\hat{y}_k = \sigma(z_k) = \frac{e^{z_k}}{\sum_k e^{z_k}}$ $k = 0, 1, 2, 3, 4$
 $\hat{y}_k = 1, 0, 1, 2, 3, 4$



Common Spatial Patterns (CSP)

Similar to PCA, maximizes variance, but CSP maximizes variance of 1 class while reducing variance of other classes. Finds all such vectors.

Algorithm

1. Consider K classes of signals $X_{k1}, X_{k2}, \dots, X_{kn}$
2. Compute Covariances of each signal

$$C_{ki} = \frac{X_{ki} X_{ki}^T}{\text{trace}(X_{ki} X_{ki}^T)} \Rightarrow \text{trace normalization}$$

3. Compute average of covariances per class

$$C_k = \frac{1}{N_k} \sum_i C_{ki}$$

4. Now we have C_1, C_2, C_3, C_4 for 4 classes

5. So to maximize variance, maximize $J(w)$

$$J(w) = \frac{w^T C_k w}{w^T C_{-k} w} \approx \frac{w^T C_k w}{w^T C_T w}$$

$$\text{where } C_k = \frac{1}{N_k} \sum_{i \in k} C_{ki}, C_T = \sum_k C_k$$

6. Find eigenvectors of $C_k w = \lambda (C_T) w$ (vii)

7. Whiten $\Rightarrow S_k = C_T^{-1/2} C_k C_T^{-1/2}$, Solve $S_k w = \lambda w$

8. Take top m & bottom m eigenvectors (Sort λ)

$$W_{MCSP} = [w_1, w_2, \dots, w_K]$$

$$X'_{ki} = W_{MCSP}^T X_{ki}$$

9. Use LDA or Multinomial Logistic regression

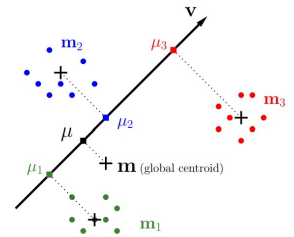
10. Concatenate X'_{ki} for all i & $k \in \{0, 1, 2, 3\}$

11. Logistic Regression $Z_k = w_k x'_{ki} + b_k$

$$\hat{y}_k = \frac{e^{Z_k}}{\sum e^{Z_k}} \text{ logits.}$$

Linear Discriminant Analysis (LDA)

$\mu_j \Rightarrow$ projection
 $m_j \Rightarrow$ mean



LDA is Supervised method to find vectors/directions that maximizes the variance across classes & minimizes the variance within the class. So it's technically maximizes between-class variance & within-class variance.

$$J(v) = \frac{\sum_j (\mu_j - \bar{\mu})^2}{\sum_j s_j^2} \Rightarrow \frac{v^T S_b v}{v^T S_w v}$$

So $\mu_j, \bar{\mu}, s_j$ are all variance across points μ_j, m, x_j projected on vector v .

$$\mu_j = v^T m_j \quad \bar{\mu} = v^T m$$

$$S_b = \sum_j (\mu_j - \bar{\mu})(\mu_j - \bar{\mu})^T \Rightarrow \text{between class}$$

$$S_w = \sum_j \sum_{x \in c_j} (x - m_j)(x - m_j)^T \Rightarrow \text{within class}$$

Theorem: Suppose S_w is non-singular, then solution to maximize $J(v)$ is given by highest eigenvectors of $S_w^{-1} S_b \Rightarrow S_w^{-1} S_b v = \lambda v$

1. Compute S_w & S_b for the points X .

2. Solve the generalized eigenvalue problem $S_b v = \lambda S_w v$
top $K \leq c-1$ eigenvectors $V_K = [v_1, v_2, \dots, v_K]$

3. Project X onto them $Z = X \cdot V_K$ & classify
 $1 \times K \quad 1 \times d \quad d \times K$

Z is low-dimensional LDA feature vector.

4. $\hat{y}_c = \arg \max_c \left(z^T \Sigma^{-1} \mu - \frac{1}{2} \mu^T \Sigma^{-1} \mu + \log p(c) \right)$ (vii)

Assuming each class c follows Gaussian with variance $\Sigma = N(Z; \mu_c, \Sigma)$

Minimum Distance to Mean (MDM)

Algorithm

1. $X_{i,a,e}$; $i \in \{1, 2, \dots, 109 \text{ subjects} \Rightarrow \text{Physionet}\}$
 $a \in \{0, 1, 2, 3\} \Rightarrow \text{actions}$
 $e \in \{1, 2, 3, \dots, N_T\}$

2. Compute Covariances $C_{i,a,e}$

$$C_{i,a,e} = \frac{1}{c-1} X_{i,a,e} X_{i,a,e}^T$$

3. Compute C_{ref} (global mean) for each class $K \in \{0, 1, 2, 3\} \Rightarrow 4 \text{ actions}$

$$\text{Euclidean Mean}(K) = \frac{1}{N} \sum_{i,e} C_{i,e,a} |_{a=K}$$

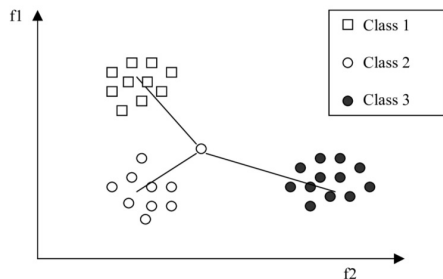
$$\text{Riemannian Mean}(K) = \arg \min_C \sum_{i,e} d_R^2(C, C_{i,e,a}) |_{a=K}$$

$$\text{Log-Euclidean Mean}(K) = \frac{1}{N} \sum_{i,e} \log C_{i,e,a} |_{a=K}$$

4. For Covariance $C_{i,e,a}$, for $K \in \{0, 1, 2, 3\}$

$$D_K(C_{i,e,a}, C_{ref,K}) = \left\| \log(C_{i,e,a}^{-1/2} C_{ref,K} C_{i,e,a}^{-1/2}) \right\|_F$$

5. $\hat{y}_K = \arg \min_K D_K(C_{i,e,a}, C_{ref,K})$



Riemann Procrustes Analysis (RPA)

This process generally has 3 steps
 → re-center → scale → rotate.

* Limitation: This is a transfer learning method.
 can't use for cross subject as we don't have labeled test data.

Source (S): train
 Test (T): test

* re-center ⇒ Riemannian Alignment.

$$C' = C_{ref}^{-1/2} C C_{ref}^{-1/2} \Rightarrow [C'_{ref} = I]$$

* scale: we need to find the dispersions between the source (train) & target (test)

$$\text{train dispersions: } \sigma_S^2 = \frac{1}{|S|} \sum d^2(C, C', I) \Rightarrow \log C'_S$$

$$\text{test dispersions: } \sigma_T^2 = \frac{1}{|T|} \sum d^2(C, C', I) \Rightarrow \log C'_T$$

$$p = \sqrt{\sigma_S^2 / \sigma_T^2} \Rightarrow \text{stretch factor}$$

↳ scalar ⇒ euclidean

$$Z = \log C', Z' = pZ, \tilde{C}' = \log Z'$$

So $\tilde{C}' \Rightarrow$ covariance with Matched Dispersion

$$\text{* rotate: } \tilde{C}_{ref, S, k} = \arg \min_C \sum_{C_S \in S, y=k} d^2(C, C_S, k)$$

$$\tilde{C}_{ref, T, k} = \arg \min_C \sum_{C_T \in T, y=k} d^2(C, C_T, k)$$

At tangent space, $\tilde{C}_{ref, T, k}$ is to be rotated (per action) aligned to $\tilde{C}_{ref, S, k}$. So the phase direction is same

$$T_{T, k} = \log(\tilde{C}_{ref, T, k}) \quad T_S = \log(\tilde{C}_{ref, S, k})$$

$$U = \underset{U}{\text{minimize}} \sum_k \|U^T T_{T, k} U - T_{S, k}\|_F^2. \text{ This update}$$

is a type of alignment, After this we can use TSLR or LDA for classification

Observations

Model	X	$C_{ref}^{-1/2} (X - X_{ref})$
CSP + LDA	42.13 ± 15.84%	45.60 ± 15.81%

Ablation Study

MODEL	ALIGNMENT			
	NA	EA	RA	LEA
MDN	37.00 ± 12.18%	50.81 ± 16.81%	52.43 ± 15.66%	52.62 ± 15.52%
TSLR	33.72 ± 6.80%	49.50 ± 12.21%	50.58 ± 12.68%	50.12 ± 12.92%
TSA + LDA	36.11 ± 8.79%	52.93 ± 15.06%	50.51 ± 15.29%	50.86 ± 15.60%
CSP + LDA	42.13 ± 15.84%	42.18 ± 15.84%	"	"
CSP + LDA whitening ($C_{ref}^{-1/2} X$)	"	41.67 ± 15.64%	42.21 ± 15.57%	41.32 ± 15.70%
CSP + LDA whitening + 2 Scale norm ($C_{ref}^{-1/2} (X - X_{ref})$)	45.60 ± 15.81%	46.18 ± 15.91%	46.60 ± 15.81% (X _{ref} not possible)	46.49 ± 15.22%

⇒ Insight: Given a Covariance C

After eigendecomposition $C = U \Lambda U^T$ where U is an orthogonal matrix ($U^T U = I$) and Λ is a diagonal matrix with diagonals being eigenvalues. $[C - \lambda I = 0]$

$$C u = \lambda u \text{ where } [u_1, u_2, \dots]$$

compose U [eigenvector] & $\lambda_1, \lambda_2, \dots$ compose diagonal elements of Λ

Covariance ⇒ ellipsoid, eigenvectors ⇒ axes & eigenvalues are the magnitude of the axes.

⇒ From RPA we can realize that eigenvectors ⇒ actions & eigenvalues ⇒ subject. But not necessarily true

Other Models based on Loss functions.

Models built on CORAL loss, MEKT, MAML & DeepCORAL all are not true zero shot. DANN can be used in true zero shot to penalize subject variance but needs subject label for accuracy improvement

⇒ Needs test data ⇒ Not true zero shot

CORAL ⇒ Domain Adaptation (Not DG)

$$L_{CORAL} = \frac{1}{4d^2} \|C_S - C_T\|^2 \quad (48-52\%)$$

MEKT ⇒ Manifold Embedded Knowledge Transfer (54-56%)

Learn a projection matrix W that

$$\min \alpha \|W^T X_S - W^T X_S L_S\|_F^2 + \beta \|W^T X_T - W^T X_T L_T\|_F^2 + \gamma \text{MMD}(W^T X_S, W^T X_T)$$

- preserves source structure (X_S).
 - preserves target manifold structure (X_T)
 - minimizes domain discrepancy (MMD)
- ⇒ maximum mean discrepancy.

(Needs Target data ⇒ does not work instantly on unseen data)

MAML ⇒ Model Agnostic Meta Learning (45%)

Each subject is a task.

Inner loop: simulates learning a subject (source)

$$\theta' = \theta - \alpha \nabla_{\theta} L_S(\theta)$$

Outer loop: Learn parameters to generalize target

$$\theta \leftarrow \theta - \beta \nabla_{\theta} L_T(\theta')$$

PRE-PROCESSING ALGORITHMS.

DCR: Discriminative Class representative Pre-Aligner

1. Input $C_{i,a,e}$
2. Riemann Alignment (RA): Compute AIRM mean & Whiten each covariance per subject

$$C'_{i,a,e} = X_{ref}^{-1/2} C_{i,a,e} X_{ref}^{-1/2}$$
3. Compute log map $L_{i,a,e} = \log(C'_{i,a,e})$
4. Compute class mean M_k & global mean M .
5. Initialize rotation & dispersion:
 - Set $A = 0$, compute $R = \exp(A - A^T)$
 - Initialize λ (scaling factor).
6. Optimization loop:
 a. Update rotation: $R = \exp(A - A^T)$
 b. Rotate all log covariances: $L_{i,a,e} = R^T L_{i,a,e} R$
 c. Compute discriminative energies.

$$B(R) = \sum_k N_k \|\text{diag}(R^T (M_k - M) R)\|_F^2$$

$$W(R) = \sum_k N_k \|\text{diag}(L_{i,a,e} - R^T M_k R)\|_F^2$$

 d. Compute centering penalty (Identity scale)

$$\text{center}(R) = \|\text{offdiag}(L_{i,a,e})\|_F^2$$

 f. Loss fn:

$$\mathcal{L}(R, \lambda) = \frac{W(R)}{B(R) + \epsilon} + \alpha(\lambda - 1)^2 + \frac{1}{N} \|R - I\|_F^2 + \lambda_c \text{center}(R)$$

 g. Take gradient step on A & λ .
7. Extract & Save optimized rotation R^*
8. Apply transform
 (i) Rotate using R^* : $L_{R^*,i,a,e} = R^{*T} L_{i,a,e} R^*$
 (ii) exp relative to I : $C'_{i,a,e} = \exp_I(L_{R^*,i,a,e})$
9. Output: $C'_{i,a,e}$
 RA + TSLR: 52.89% DCR + TSLR: 53.01%
 RA + MDM: 52.43% DCR + MDM: 50.46%
 RA + TSA-LDA: 53.51% DCR + TSA-LDA: 53.97%

RIFU: Riemann Fisher U-Net Pre-Aligner

Input: $C_{i,a,e} \in \mathbb{R}^{c \times c}$ (SPD), $y_{i,act}$, subject ids

Riemann Alignment: $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$

SPD Linear layer: (My idea $\Rightarrow$ No published paper)

Weight matrices $W \in \mathbb{R}^{c \times d_{hidden}} / \mathbb{R}^{c \times d_{out}}$.

Residual connections: We need to merge input from previous layer with the current input. Similar to addition.

Consider C_1 from previous layer & C_2 from current layer we can merge them before sending input

$$C_{in} = \text{merge}(C_1, C_2) = \exp_I\left(\frac{1}{2}(\log_I C_1 + \log_I C_2)\right)$$

$C_{in} \Rightarrow$ SPD matrix.

U-Net [Encoder-Decoder Architecture]

$$\Phi^E = W^T C'_{i,a,e} W; W \in \mathbb{R}^{c \times d}, d \leq c \text{ (encoder)}$$

$$\Phi^D = W^T C_{enc} W; W \in \mathbb{R}^{c \times d}, d \geq c \text{ (decoder)}$$

$$V(C) = \Phi^{E(n)}(\text{merge}(\Phi^{E(n-1)}(\dots \Phi^{E(1)}(C)) \dots))$$

$$E(U) = \Phi^{D(n)}(\text{merge}(\Phi^{D(n-1)}(\dots \Phi^{D(1)}(U)) \dots))$$

$$E(U) = \text{Symmetric } \underline{C + \epsilon I} \Rightarrow \text{eigenvalues} \Rightarrow \text{positive definite}$$

vectorize $E(U)$ & mean $\text{vec}_A(L), y_1, y_2, \dots$

Compute mean per action, of action means, per subject

$$W(a) = \frac{1}{N_a} \sum_i (y_{i,a} - m_a)(y_{i,a} - m_a)^T$$

$$B(a) = \frac{1}{N_{am}} \sum_i (m_a - m)(m_a - m)^T$$

$$W(s) = \frac{1}{N_s} \sum_i (y_{i,s} - m_s)(y_{i,s} - m_s)^T$$

$$\text{Recon} = \frac{1}{Z_d} \sum_i \|z_i - z_i^m\|_2^2$$

Training loop:

- a. Sample covariances. $C_{i,a,e} \in \mathbb{R}^{c \times c}$
- b. Forward pass & obtain $E(V(C_{i,a,e}))$
- c. Vectorize all $E(VCC)$

d. Compute global, per action, per subject means.

e. Compute loss terms: $W(a), B(a), W(s), \text{Recon}$.

$$\mathcal{L}_{RIFU} = \lambda_{\text{within act}} W(a) - \lambda_{\text{bet act}} B(a) - \lambda_{\text{sub}} W(s) + \lambda_{\text{recon}} \text{Recon}$$

f. Take gradient w.r.t. W to update all learnable params W .

Store best model parameters according to validation

Transform:

a. Compute log map $L_{i,a,e} = \log(C_{i,a,e})$ (for test)

b. Compute $C'_{i,a,e} = E^*(V^*(L_{i,a,e}))$

c. Use $C'_{i,a,e, \text{out}} = C'_{i,a,e}$ as the pre-aligned covariances with preprocessing done.

Output: Trained SPD parameters (W) & $C'_{i,a,e, \text{out}}$.

Observation:

RA + TS-SVM-RBF: 54.98%

RA + TSLR: 52.89%

RA + MDM: 52.43%

RA + TSA-LDA: 53.51%

RIFU + TS-SVM-RBF: 55.17%

RIFU + TSLR: 54.78%

RIFU + MDM: 52.51%

RIFU + TSA-LDA: 54.24%

CLASSIFIERS

RiFUNetClassifier

1. Input: Covariances $C_{i,a,e}$, $\hat{y}_{act}$, $\hat{y}_{sub}$.
2. Riemann Alignment: $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$.
3. We discussed SPD U-Net above using congruence transform
 $\Phi_{SPD} = W^T C_{i,a,e} W \Rightarrow W \in \mathbb{R}^{c \times d}$ $\begin{cases} d < c; \text{encoder} \\ d > c; \text{decoder} \end{cases}$
4. Forward Step through RiFUNet:
 (a) Compute $C_{out,i,a,e} = E(U(C'_{i,a,e}))$ using SPD U-Net.
 (Congruence Transform + Residual Connections)
5. Tangent Space Projection:
 (a) Once we get C_{out} , compute logmap $L_i = \log(C_{out,i,a,e})$
 (b) $\text{vec}_{\Delta}(L_i) = \Sigma_i$
6. Initiate action head
7. Compute loss components (like before)
 (a) within action $W(A)$
 (b) between action $B(A)$
 (c) within subject $W(S)$
 (d) predict action using action head & compute L_{CE}^{act}
 (e) reconstruction loss: $R = \text{Recon}(C_{i,a,e}, C_{out})$
8. Compute loss

$$L = \lambda_f (\lambda_{\text{within act}} W(A) - \lambda_{\text{bet act}} B(A) - \lambda_{\text{within sub}} W(S)) + \lambda_{CE} L_{CE}^{act} + \lambda_{recon} R$$
9. Backpropagate & update learning parameters. RiFUNet, action head & subject head
10. Output: $\hat{y}_{pred, action}$

SPDNetClassifier : Deep Congruence Networks

1. Input: $C_{i,a,e}$, $\hat{y}_{act}$, $\hat{y}_{sub}$, λ_{within} , λ_{between} , λ_{act} , λ_{sub}
2. Riemann Alignment: $C'_{i,a,e} = C_{ref}^{-1/2} C_{i,a,e} C_{ref}^{-1/2}$
3. SPD-Net forward pass:
 for each layer l : $C^{(l+1)} = W^T C^{(l)} W$
 obtain final SPD output = $C_{i,a,e}^{out}$
4. Tangent space feature extraction:
 $L_{i,a,e} = \log(C_{i,a,e}^{out})$ $\Sigma_i = \text{vec}_{\Delta}(L_{i,a,e})$
5. Compute fisher ratio action terms
 $\mu = \frac{1}{n_a} \sum \mu_a$
 (a) compute action means μ_a & global mean
 (b) compute within act $W(a)$ & between act $B(a)$
6. Compute fisher subject terms
 (a) compute subject mean μ_s & global mean μ
 (b) Compute within subject $W(s)$ & between subject $B(s)$
7. Compute Cross Entropy Loss
 Compute $\hat{y}_i^{act}$ using action head & compute L_{CE} .
8. Compute Loss:

$$L = \lambda_{CE} L_{CE} + \lambda_{action} (\lambda_{\text{within}} W(a) - \lambda_{\text{between}} B(a)) + \lambda_{subject} (\lambda_{\text{between}} B(s) - \lambda_{\text{within}} W(s))$$
9. Output: $\hat{y}_{pred, action}$

Assumption: Initiate W as orthonormal ($W^T W = I$) to retain rotate functionality & full rank properly. Note: Congruence transforms don't necessarily need W to be orthogonal. However $lr = 1e-3$ and initiating this proved to boost $Loss$ by 0.5%. Since $lr = 1e-3$, very slight deviation. So retains orthonormality

APPENDIX

APPENDIX

(i) symmetry of geodesic distance

$$d(C_1, C_2) = \|\log C C_1^{-1/2} C_2 C_1^{-1/2}\|_F$$

$$d(C_2, C_1) = \|\log C C_2^{-1/2} C_1 C_2^{-1/2}\|_F$$

$$\text{let } X = C_1^{-1/2} C_2 C_1^{-1/2}$$

$$(AB)^{-1} = B^{-1}A^{-1}$$

$$X^{-1} = (C_1^{-1/2} C_2 C_1^{-1/2})^{-1}$$

$$= (C_2 C_1^{-1/2})^{-1} (C_1^{-1/2})^{-1}$$

$$= (C_1^{-1/2})^{-1} C_2^{-1} (C_1^{-1/2})^{-1}$$

$$= C_1^{1/2} C_2^{-1} C_1^{1/2}$$

$$X^{-1} = C_1^{1/2} (C_2^{-1} C_1) C_1^{-1/2}$$

$$X = C_1^{1/2} (C_1^{-1} C_2) C_1^{-1/2}$$

$$\text{eig}(X^{-1}) = \text{eig}(C_2^{-1} C_1)$$

$$\text{eig}(X) = \text{eig}(C C_1^{-1} C_2)$$

Property: if $A = PAP^{-1}$, $\text{eig}(A) = \text{eig}(PAP^{-1})$

$$(C_2^{-1} C_1)^{-1} = C_1^{-1} C_2$$

$$\therefore \text{eig}(C C_2^{-1} C_1) = \text{reciprocal}(\text{eig}(C_1^{-1} C_2))$$

$$\|\log X\|_F = \left(\sum_i (\log \lambda_i)^2 \right)^{1/2}$$

$$\|\log X^{-1}\|_F = \left(\sum_i (\log \lambda_i^{-1})^2 \right)^{1/2} = \left(\sum_i (\log \lambda_i)^2 \right)^{1/2}$$

$$= \|\log X\|_F$$

$$\therefore \|\log X^{-1}\| = \|\log X\|_F \Rightarrow d(C_1, C_2) = d(C_2, C_1)$$

$$(ii) \text{ Euclidean Mean} = \underset{C}{\text{argmin}} \sum_{i,a} d_{LE}^2(C, C_{i,a})$$

$$\nabla_C \sum_i d_{LE}^2(C, C_i) = 2 \sum_{i,a} (C - C_{i,a}) = 0$$

$$C_{E, \text{ref}} = \frac{1}{N_T} \sum_{i,a} C_{i,a}$$

$$(iii) \text{ log-Euclidean Mean} = \log \left(\underset{C}{\text{argmin}} \sum_i d_{LE}^2(C, C_i) \right)$$

$$\nabla_C \sum_i d_{LE}^2(C, C_i) = 2 \sum_{i,a} (\log C - \log C_{i,a}) (C^{-1}) = 0$$

$$C_{LE, \text{ref}} = \log C = \frac{1}{N_T} \sum_{i,a} \log C_{i,a}$$

$$(iv) C_e^{-1} = C_{R, \text{ref}, i}^{-1/2} C_e C_{R, \text{ref}, i}^{-1/2}$$

Proof

$$C_{R, \text{ref}, i} = \text{SPD matrix. } C_{\text{ref}} = C_{\text{ref}}^T$$

We know $C_{R, \text{ref}, i}$ is unique for each i

So to generalize across a $i \in \{1, 2, 3, \dots, 1093\}$

$$C_{R, \text{ref}, 1} = C_{R, \text{ref}, 2} = \dots = C_{R, \text{ref}, 1093} = I$$

$$C_{R, \text{ref}, i} = I \text{ (All eigenvectors (axes) } \Rightarrow \text{unity)}$$

$$W C_{R, \text{ref}, i} W^T = I.$$

$$W = W^T = C_{R, \text{ref}, i}^{-1/2}$$

$$C_{R, \text{ref}, i}^{-1/2} C_{R, \text{ref}, i} C_{R, \text{ref}, i}^{-1/2} = I \text{ for all } i$$

$$C_{R, \text{ref}, i} = \underset{C}{\text{argmin}} \sum_e d_R^2(C, C_e)$$

$$= \underset{C}{\text{argmin}} \sum_e \|\log(C^{-1/2} C_e C^{-1/2})\|_F = C_{\text{ref}}$$

$$= \underset{C}{\text{argmin}} \sum_e \|\log(C^{1/2} C_{\text{ref}}^{-1/2} C_e C_{\text{ref}}^{-1/2} C^{-1/2})\|_F$$

$$= \underset{C}{\text{argmin}} \sum_e \|\log(C^{-1/2} C_{\text{ref}}^{-1/2} C_e C_{\text{ref}}^{-1/2} C^{-1/2})\|_F$$

$$= \underset{C}{\text{argmin}} \sum_e \|\log((C_{\text{ref}} C)^{-1/2} C_e (C_{\text{ref}} C)^{-1/2})\|_F$$

$$= \underset{C_{\text{ref}}}{\text{argmin}} \sum_e \|\log((C_{\text{ref}} C)^{-1/2} C_e (C_{\text{ref}} C)^{-1/2})\|_F$$

$$C_{\text{ref}}; C_{\text{ref}} = C_{\text{ref}} C \Rightarrow C_{\text{ref}} = I \text{ or } C = I$$

$\Rightarrow$ We know at this condition $C_{\text{ref}} = C_{\text{ref}}$.

$$\therefore C = I. \Rightarrow C_{\text{ref}} = I$$

We shaped all covariances such that we $C_{\text{ref}} = I$ (all eigenvectors are unit vectors)

(v) Signal whitening $\Rightarrow$ linear transformation

$$C_{i,a,e}^{-1} = C_{R, \text{ref}, i}^{-1/2} C_{i,a,e} C_{R, \text{ref}, i}^{-1/2}$$

$$= C_{R, \text{ref}, i}^{-1/2} \frac{1}{N-1} X_{i,a,e} X_{i,a,e}^T C_{R, \text{ref}, i}^{-1/2}$$

$$= \frac{1}{N-1} C_{R, \text{ref}, i}^{-1/2} X_{i,a,e} X_{i,a,e}^T C_{R, \text{ref}, i}^{-1/2}$$

$$= \frac{1}{N-1} (C_{R, \text{ref}, i}^{-1/2} X_{i,a,e}) (C_{R, \text{ref}, i}^{-1/2} X_{i,a,e})^T$$

$$= \frac{1}{N-1} X_{i,a,e}' X_{i,a,e}'^T$$

$$X_{i,a,e}' = C_{R, \text{ref}, i}^{-1/2} X_{i,a,e}$$

$$C \times C. C \times T \Rightarrow C \times T$$

$$\text{So } U_{\text{ref}} \Lambda_{\text{ref}}^{-1/2} U_{\text{ref}}^T X_{i,a,e}$$

rotate back $\left[\begin{array}{c} \text{transform} \\ \text{rotate signal} \end{array} \right] \Rightarrow$ linear transform

We normalize the signal to the reference coordinates.

(vi) $C_K \omega = \lambda (C_E) \omega$
 $C_E^{-1/2} C_K C_E^{-1/2} \omega = \lambda (C_E^{-1/2} C_E C_E^{-1/2}) \omega$
 $S_K \omega = \lambda \omega$
 $S_K = C_E^{-1/2} C_K C_E^{-1/2}$
 Let eigendecomposition of $C_E = E \Lambda E^T$
 $C_E^{-1/2} = E \Lambda^{-1/2} E^T$
 eigendecomposition of $S_K = U \Lambda U^T$; $U^T U = I$
 $S_K = U \Lambda U^{-1} \Rightarrow U^{-1} S_K U = U^T S_K U = \Lambda$
 $U^T C_E^{-1/2} C_K C_E^{-1/2} U = \Lambda$ [$C_E^{-1/2} = (C_E^{-1/2})^T$]
 $(C_E^{-1/2} U)^T C_K (C_E^{-1/2} U) = \Lambda$ [diagonal]
 $W^T C_K W = \Lambda$, $W^T C_E W = I$
 $W^T = W^{-1} C_E^{-1} \Rightarrow W^{-1} C_E^{-1} C_K W = \Lambda$
 $C_K W = \lambda C_E W \Rightarrow$ this is verification
 $\therefore W = C_E^{-1/2} U$ where U is eigenvector of whitened covariance

(vii) Assume C classes, class c has Gaussian distribution. $N(x_c; \mu_c, \Sigma)$ with same Σ

Bayes theorem:

$$P(y = c/x) \propto P(x/y = c) P(y = c)$$

$$P(y = c) = \pi_c \text{ (prior)}$$

$$P(y = c/x) \propto P(x/y = c) \cdot \pi_c \quad (1)$$

$$x|y = c \sim N(\mu_c, \Sigma)$$

$$P(x/y = c) = \frac{1}{\sqrt{2\pi}\Sigma} e^{-\frac{(x-\mu_c)^T \Sigma^{-1} (x-\mu_c)}{2}}$$

From (1): $\log P(y = c/x) = \log(P(x/y = c)) + \log \pi_c$

$$\log(P(y = c/x)) = \log\left(\frac{1}{\sqrt{2\pi}\Sigma} e^{-\frac{(x-\mu_c)^T \Sigma^{-1} (x-\mu_c)}{2}}\right) + \log \pi_c$$

$$= -\frac{1}{2}(x-\mu_c)^T \Sigma^{-1} (x-\mu_c) - \frac{1}{2} \log 2\pi \Sigma + \log \pi_c$$

$$= -\frac{1}{2} x^T \Sigma^{-1} x + \frac{1}{2} x^T \Sigma^{-1} \mu_c + \frac{1}{2} \mu_c^T \Sigma^{-1} x - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c - \frac{1}{2} \log 2\pi \Sigma + \log \pi_c$$

$$= -\frac{1}{2} [x^T \Sigma^{-1} x - 2 x^T \Sigma^{-1} \mu_c + \mu_c^T \Sigma^{-1} \mu_c + \log 2\pi \Sigma + \log \pi_c]$$

We need to maximize $P(y = c/x) \propto \log P(y = c/x)$
 $\Rightarrow$ depends only on class c (changing factor)
 $x^T \Sigma^{-1} x$, $\log 2\pi \Sigma$ are constant $\Rightarrow$ independent of c .
 maximize $\Rightarrow x^T \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \log \pi_c$
 C

$$C = \operatorname{argmax}_c \left(x^T \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \log \pi_c \right)$$

Case LDA: $x^T V_K = x_K^T$. Otherwise the $\mu_c \Rightarrow$ distribution of $x_K^T \sim N(x_K; \mu_{cK}, \Sigma_K)$

$$C = \operatorname{argmax}_c \left(x_K^T \Sigma_K^{-1} \mu_{cK} - \mu_{cK}^T \Sigma_K^{-1} \mu_{cK} + \log \pi_{cK} \right)$$

(viii) if C = symmetric (PSD) positive semi definite. $\Rightarrow W^T C W \geq 0$ is the condition as W is initiated orthonormal.
 So if $C_{out} = W^T C W$.

For $x \geq 0$: $x^T C_{out} x \geq 0$ [SPD]
 $x^T (C_{out} + \epsilon I) x = x^T C_{out} x + \epsilon \|x\|^2 > 0$

Methods to maintain W as orthonormal
 orthonormal $\Rightarrow W^T W = I$

Currently let $M = W^T W \neq I$.

So how to make W orthonormal while maintaining relationship with gradient
 Let Wx be orthonormal

$$(Wx)^T (Wx) = I \Rightarrow x^T W^T W x = I$$

So $x^T (W^T W) x = I$, Here we need to make W orthogonal by congruence transform.
 $x^T M x = I \Rightarrow x = M^{-1/2}$ as $W^T W$ is symmetric $\Rightarrow \boxed{x = M^{-1/2}}$

So $W_{orth} = W M^{-1/2} = W (W^T W)^{-1/2}$
 Here we are trying to project the non orthonormal W onto the Stiefel Manifold
 $S(n, p) = \{C \in \mathbb{R}^{n \times p} ; C^T C = I\}$.

(vi) maximize $J(\omega) = \frac{\omega^T C_K \omega}{\omega^T C_E \omega} = \frac{\omega^T C_K \omega}{\omega^T C_E \omega}$

Here we need to constraint $\omega^T C_E \omega = 1$ & maximize $\omega^T C_K \omega$. So we write Lagrangian

$$L = \omega^T C_K \omega - \lambda (\omega^T C_E \omega - 1)$$

$$\frac{dL}{d\omega} = C_K \omega - \lambda C_E \omega = 0$$

$$C_K \omega = \lambda C_E \omega$$

We try to maximize λ through this L .
 So basically the highest eigenvalue corresponds to the best vector

